

# EF Core. Подход Database-First. Фильтрация данных.

Entity Framework Core позволяет разрабатывать приложение по подходу Database-First, где приложение разрабатывается на основе существующей базы данных.

В данной работе рассмотрим этот подход. И будем работать с фильтрацией данных.

## Подключение к существующей базе данных. Генерация моделей

Первым делом создадим пустой WPF-проект и установим необходимые библиотеки:

- `Microsoft.EntityFrameworkCore`
- `Microsoft.EntityFrameworkCore.Tools`
- `Microsoft.EntityFrameworkCore.SqlServer`

Для работы на платформе .NET 8 версия пакетов 9.0.11

Далее можно подключаться к базе данных.

Для примера используется база данных с одной таблицей из следующего репозитория:

[FakeFormData](#)

Данная БД имеет одну таблицу `Form`

| SERVER.demo_...t - dbo.Form |               |               |                          |
|-----------------------------|---------------|---------------|--------------------------|
|                             | Имя столбца   | Тип данных    | Разрешить знач...        |
| 🔑                           | Id            | int           | <input type="checkbox"/> |
|                             | Name          | nvarchar(100) | <input type="checkbox"/> |
|                             | Last_Name     | nvarchar(100) | <input type="checkbox"/> |
|                             | Age           | int           | <input type="checkbox"/> |
|                             | Birthday_Year | int           | <input type="checkbox"/> |
|                             | Height        | int           | <input type="checkbox"/> |
|                             | Weight        | float         | <input type="checkbox"/> |

При импорте БД из скрипта, на ПК в аудитории, не забывайте менять имя базы данных

Далее после вставки данных в таблицу (всего 10000 строк, запрос для проверки: `SELECT COUNT(Id) as 'count' FROM FORM`), можно настроить подключение проекта к базе данных,

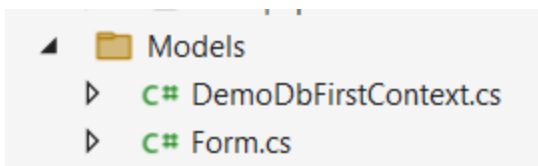
для этого нужно выполнить следующую команду:

```
Scaffold-DbContext 'Server=sql.ects;Database=demo_db_first;User
Id=*****;Password=*****;TrustServerCertificate=True;'
Microsoft.EntityFrameworkCore.SqlServer -o Models
```

Строка подключения идентична строке, которую вы создаете самостоятельно в классе контекста предыдущих работ!!!

- `Scaffold-DbContext` - утилита пакета EF Core, которая подключается к БД по строке подключения
- `Microsoft.EntityFrameworkCore.SqlServer` - используемый драйвер для подключения к БД
- `-o Models` - папка в проекте, куда будут помещены сгенерированные модели и контекст

В результате EF Core сгенерирует класс контекста и классы моделей на основании базы данных, со всеми навигационными свойствами



Models/Form.cs

```
public partial class Form
{
    public int Id { get; set; }
    public string Name { get; set; } = null!;
    public string LastName { get; set; } = null!;
    public int Age { get; set; }
    public int BirthdayYear { get; set; }
    public int Height { get; set; }
    public double Weight { get; set; }
}
```

Models/DemoDbFirstContext.cs

```
public partial class DemoDbFirstContext : DbContext
{
    public DemoDbFirstContext()
    {
    }
}
```

```

public DemoDbFirstContext(DbContextOptions<DemoDbFirstContext> options)
    : base(options)
{
}

public virtual DbSet<Form> Forms { get; set; }

protected override void OnConfiguring(DbContextOptionsBuilder
optionsBuilder)
#warning To protect potentially sensitive information in your connection
string, you should move it out of source code. You can avoid scaffolding the
connection string by using the Name= syntax to read it from configuration -
see https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on
storing connection strings, see https://go.microsoft.com/fwlink/?LinkId=723263.
=>
optionsBuilder.UseSqlServer("Server=sql.ects;Database=demo_db_first;User
Id=*****;Password=*****;TrustServerCertificate=True;");

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Form>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("PK__Form__3214EC073C464568");

        entity.ToTable("Form");

        entity.Property(e =>
e.BirthdayYear).HasColumnName("Birthday_Year");
        entity.Property(e => e.LastName)
            .HasMaxLength(100)
            .HasColumnName("Last_Name");
        entity.Property(e => e.Name).HasMaxLength(100);
    });

    OnModelCreatingPartial(modelBuilder);
}

partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}

```

Комментарий #warning , говорит что строка подключения содержит чувствительные данные (логин/пароль БД), и не рекомендует хранить строку в коде. На работу приложения не влияет

На этом все. Дальше с БД можно работать как и раньше, только код моделей и контекст EF Core сгенерировал самостоятельно.

## Реализация макета окна и отображение списка из базы данных

Реализуем вывод информации из БД на экран, но предварительно установим в проект библиотеку `MaterialDesignTheme` чтобы интерфейс программы выглядел современно.

В данном примере не будут рассматриваться операции добавления, изменения и удаления, поэтому будет идти работа с одним окном.

Для начала настроим Material Design и установим нужные цвета в `App.xaml` :

```
<Application.Resources>
  <ResourceDictionary>
    <ResourceDictionary.MergedDictionaries>
      <ResourceDictionary
Source="pack://application:,,,/MaterialDesignThemes.Wpf;component/Themes/MaterialDesign2.Defaults.xaml" />
      <materialDesign:BundledTheme BaseTheme="Light"
PrimaryColor="DeepPurple" SecondaryColor="Lime"/>
      <ResourceDictionary>
        <SolidColorBrush x:Key="MaterialDesign.Brush.Primary"
Color="#00fa9a"/>
        <SolidColorBrush
x:Key="MaterialDesign.Brush.Primary.Foreground" Color="#000"/>
        <SolidColorBrush x:Key="MaterialDesign.Brush.Secondary"
Color="#7FFF00" />
        <SolidColorBrush
x:Key="MaterialDesign.Brush.Secondary.Foreground" Color="#fff" />
      </ResourceDictionary>
    </ResourceDictionary.MergedDictionaries>
  </ResourceDictionary>
</Application.Resources>
```

Далее реализуем форму окна и настроим привязки. В примере будем работать с моделью `Form` , в которой есть следующие поля:

- `Name` - имя
- `LastName` - фамилия
- `Age` - возраст
- `BirthdayYear` - год рождения
- `Height` - рост

- Weight - БЕС

```
<Window x:Class="_02.EFCore.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"

        xmlns:materialDesign="http://materialdesigninxaml.net/winfx/xaml/themes"
        xmlns:local="clr-namespace:_02.EFCore"
        mc:Ignorable="d"
        DataContext="{Binding RelativeSource={RelativeSource Self}}"
        Title="Данные формы" Height="450" Width="800">
    <Grid Margin="5">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="200"/>
            <ColumnDefinition/>
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="70"/>
            <RowDefinition/>
            <RowDefinition Height="30"/>
        </Grid.RowDefinitions>

        <StackPanel Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="2">
            <Label>Поиск</Label>
            <TextBox materialDesign:HintAssist.Hint="Введите текст для
поиска"/>
        </StackPanel>
        <StackPanel Grid.Row="1" Grid.Column="0">
            <Label FontSize="16">Сортировка</Label>
            <ComboBox>
                <ComboBox.Items>
                    <ComboBoxItem Tag="Age">По возрасту</ComboBoxItem>
                    <ComboBoxItem Tag="Weight">По весу</ComboBoxItem>
                    <ComboBoxItem Tag="Heigth">По росту</ComboBoxItem>
                </ComboBox.Items>
            </ComboBox>
            <Label>Фильтры</Label>
            <StackPanel>
                <Label FontSize="16">Фильтр по росту</Label>
                <Label>от</Label>
                <TextBox />
                <Label>до</Label>
                <TextBox />
            </StackPanel>
        </StackPanel>
    </Grid>
</Window>
```

```

        <Button Margin="5">Сбросить</Button>
    </StackPanel>
    <materialDesign:ColorZone Grid.Row="1" Grid.Column="1"
        Mode="SecondaryMid">
        <ListView HorizontalContentAlignment="Stretch">
            <ListView.ItemTemplate>
                <DataTemplate>
                    <Grid>
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="*" />
                            <ColumnDefinition Width="Auto" />
                        </Grid.ColumnDefinitions>
                        <StackPanel Grid.Column="0">
                            <TextBlock Text="{Binding LastName}" />
                            <TextBlock Text="{Binding Name}" />
                            <TextBlock>
                                <Run Text="Возраст " />
                                <Run Text="{Binding Age}" />
                            </TextBlock>
                            <TextBlock>
                                <Run Text="Год рождения " />
                                <Run Text="{Binding BirthdayYear}" />
                            </TextBlock>
                        </StackPanel>
                        <StackPanel Grid.Column="1">
                            <TextBlock HorizontalAlignment="Right">
                                <Run Text="Пост " />
                                <Run Text="{Binding Height}" />
                            </TextBlock>
                            <TextBlock HorizontalAlignment="Right">
                                <Run Text="Вес " />
                                <Run Text="{Binding Weight, StringFormat=
{{{0:F2}}}" />
                            </TextBlock>
                        </StackPanel>
                    </Grid>
                </DataTemplate>
            </ListView.ItemTemplate>
        </ListView>
    </materialDesign:ColorZone>

    <WrapPanel Grid.Row="2" Grid.ColumnSpan="2">
        <TextBlock />
        <TextBlock />
    </WrapPanel>

```

```
</Grid>
</Window>
```

Интерфейс разделен на 4 части:

- Верхняя строка, для поиска по имени
- Левая панель, содержит UI для фильтрации
- Правая часть, для отображения списка
- Нижняя панель, для отображения состояния (количество записей и количество отфильтрованных)

Чтобы избавиться от ошибок привязки изменим стили для элементов ввода `TextBox` и `ComboBox` в `App.xaml`

```
<Style TargetType="TextBox" BasedOn="{StaticResource MaterialDesignTextBox}">
    <Setter Property="AutomationProperties.Name" Value="" />
</Style>
<Style TargetType="ComboBox" BasedOn="{StaticResource
MaterialDesignComboBox}">
    <Setter Property="AutomationProperties.Name" Value="" />
</Style>
```

Далее реализуем создание контекста через паттерн `Singleton`

`Service/DBService.cs`

```
public class DBService
{
    private DemoDbFirstContext context;
    public DemoDbFirstContext Context => context;

    private static DBService? instance;

    public static DBService Instance
    {
        get
        {
            if (instance == null)
                instance = new DBService();
            return instance;
        }
    }

    private DBService() {
        context = new DemoDbFirstContext();
    }
}
```

```
}  
}
```

Далее в Code-behind окна реализуем функцию `LoadList` которая будет загружать список из базы данных

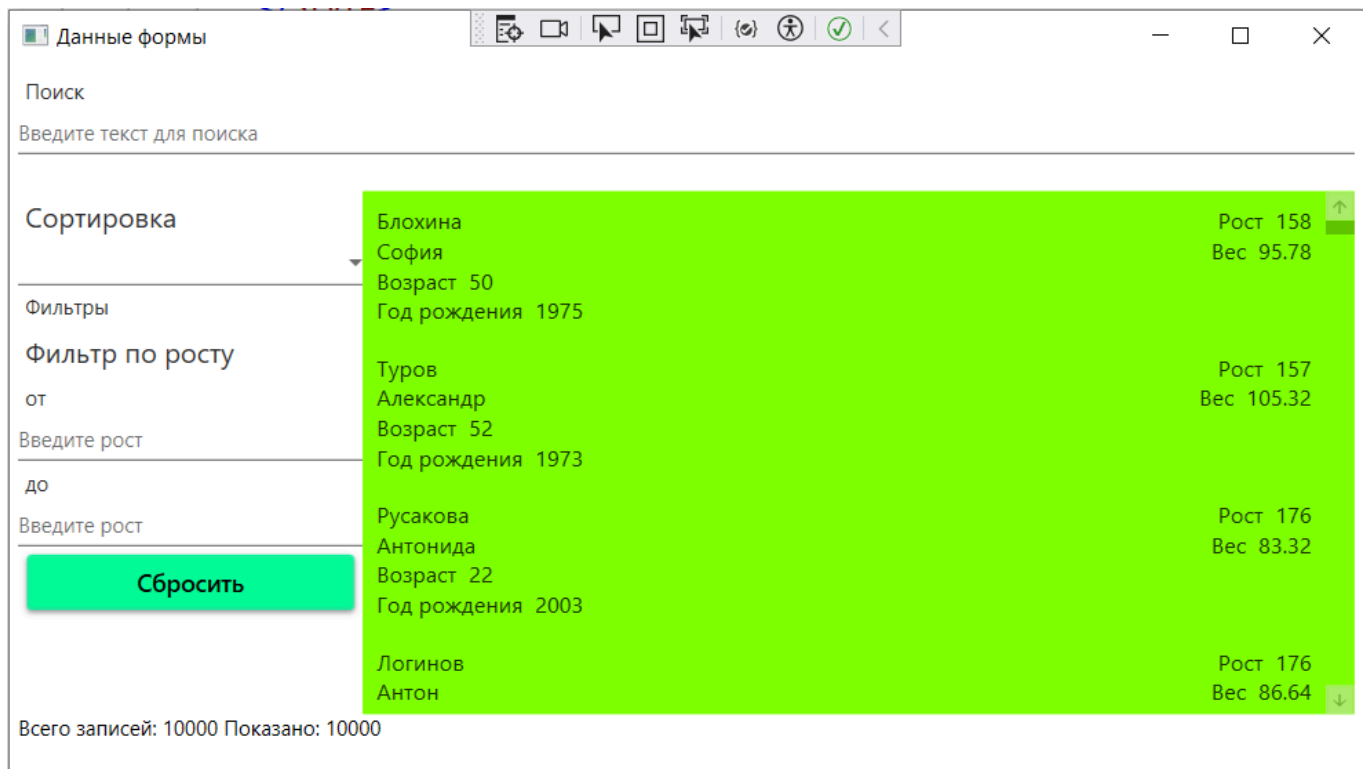
```
public DemoDbFirstContext db = DBService.Instance.Context;  
  
public ObservableCollection<Form> forms { get; set; } = new();  
  
public void LoadList(object sender, EventArgs e)  
{  
    forms.Clear();  
    foreach (var form in db.Forms.ToList())  
        forms.Add(form);  
}
```

Далее зададим `ItemsSource` для `ListView` и установим в качестве свойства `Activated` для окна функцию `LoadList`. Таким образом, каждый раз, когда окно будет становится активным, будет загружаться список из базы данных.

```
<Window x:Class="_02.EFCore.MainWindow"  
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"  
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"  
  
        xmlns:materialDesign="http://materialdesigninxaml.net/winfx/xaml/themes"  
        xmlns:local="clr-namespace:_02.EFCore"  
        mc:Ignorable="d"  
        DataContext="{Binding RelativeSource={RelativeSource Self}}"  
        Title="Данные формы" Height="450" Width="800" Activated="LoadList">  
    <!--....-->  
    <ListView ItemsSource="{Binding forms}" Name="FormsList"  
        HorizontalContentAlignment="Stretch">  
    <!--....-->  
</Window>
```

Теперь если запустить приложение, то отобразится список из базы данных





## Фильтрация данных

Для фильтрации данных можно использовать различные способы. Например запросы LINQ на базе которых EF Core будет строить запросы к базе данных. При работе с большими данными где множество связей это будет верный вариант. Но в текущем случае, постоянно дергать базу данных запросами не является хорошим вариантом, так как данные уже загружены. Поэтому будем использовать `ICollectionView`.

`ICollectionView` - это интерфейс, который представляет в коллекциях функциональные возможности управления текущими записями, пользовательской сортировки, фильтрации и группирования.

Так как в проекте для отображения списка используется `ListView`, он автоматически реализует данный интерфейс и связывает его с `ObservableCollection`. Поэтому будем использовать коллекцию `forms` для получения экземпляра `ICollectionView`.

Расширим класс окна

MainWindows.xaml.cs

```
//...
public ObservableCollection<Form> forms { get; set; } = new();
public ICollectionView formsView { get; set; }

public MainWindow()
{
```

```

        formsView = CollectionViewSource.GetDefaultView(forms);
        InitializeComponent();
    }
    //...

```

Далее реализуем поиск по имени

Так как контекстом данных является текущее окно, добавим в нем свойство для поиска и привяжем его к текстовому полю ввода:

MainWindows.xaml.cs

```

///...
public string searchQuery { get; set; } = null!;
///...

```

MainWindows.xaml

```

<TextBox Text="{Binding searchQuery, UpdateSourceTrigger=PropertyChanged}"
        materialDesign:HintAssist.Hint="Введите текст для поиска"/>

```

Ставим режим обновления привязки - `PropertyChanged`, чтобы поиск по коллекции выполнялся в момент ввода данных в текстовое поле.

Далее нужно реализовать логику поиска. Реализуем в коде функцию для поиска

MainWindows.xaml.cs

```

public bool FilterForms(object obj)
{
    if (obj is not Form)
        return false;

    var form = (Form)obj;

    if (searchQuery != null && !form.Name.Contains(searchQuery,
        StringComparison.CurrentCultureIgnoreCase))
        return false;

    return true;
}

```

Данная функция применяется для каждого объекта в коллекции. Она, проверяет, является ли объект экземпляром нужного класса, если нет - функция возвращает `false`. Далее проверяет, есть ли строка для поиска и если поле объекта не имеет вхождения по поисковой строке возвращает `false` в остальных случаях - `true`.

Далее нужно привязать данную функцию фильтрации к объекту интерфейса `ICollectionView`. Для этого внесем небольшое изменение в конструктор окна

MainWindows.xaml.cs

```
public MainWindow()
{
    formsView = CollectionViewSource.DefaultView(forms);
    formsView.Filter = FilterForms;
    InitializeComponent();
}
```

Далее нужно реализовать применение фильтров после внесения изменений. Для этого есть два варианта - реализовать для окна `INotifyPropertyChanged` или реализовать событие `TextChanged` для `TextBox`. Сейчас мы воспользуемся вторым вариантом.

MainWindows.xaml.cs

```
private void TextBox_TextChanged(object sender, TextChangedEventArgs e)
{
    formsView.Refresh();
}
```

Также привяжем этот обработчик к `TextBox` поиска

MainWindow.xaml

```
<TextBox Text="{Binding searchQuery, UpdateSourceTrigger=PropertyChanged}"
          TextChanged="TextBox_TextChanged"
          materialDesign:HintAssist.Hint="Введите текст для поиска"/>
```

Дополнительно реализуем вывод информации о количестве записей в базе данных и найденных записей.

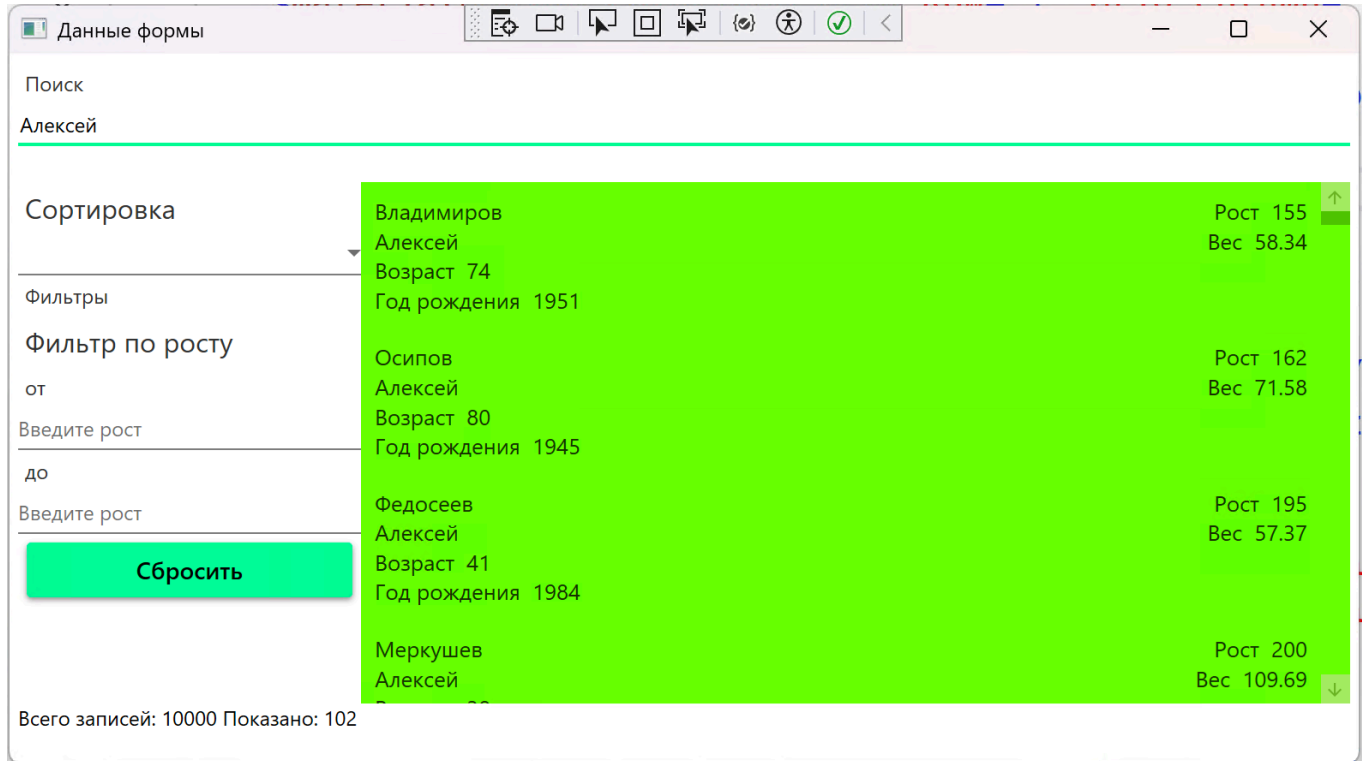
Для получения количества записей в базе, можно привязать свойство коллекции `Count`. А для количества отфильтрованных записей будем использовать элемент `ListView` и зададим ему имя `FormsList` и привяжем его свойство `Items.Count`

```

<WrapPanel Grid.Row="2" Grid.ColumnSpan="2">
    <TextBlock Text="{Binding forms.Count, StringFormat='{}Всего записей: {0}'}"/>
    <TextBlock Text="{Binding Items.Count, ElementName=FormsList, StringFormat='{} Показано: {0}'}"/>
</WrapPanel>

```

Теперь на окне будет отображаться общее количество записей и количество отображаемых (найденых) записей



Далее по аналогии с поиском будет реализован фильтр по росту - создаются свойства в `MainWindows.xaml.cs`, в code-behind устанавливается привязка, расширяется функция фильтрации и добавляется обработчик события на `TextBox` - `TextChanged`

`MainWindow.xaml.cs`

```

public string filterHeightFrom { get; set; } = null!;
public string filterHeightTo { get; set; } = null!;
//...
public bool FilterForms(object obj)
{
    if (obj is not Form)
        return false;

    var form = (Form)obj;

```

```

        if (searchQuery != null && !form.Name.Contains(searchQuery,
StringComparison.CurrentCultureIgnoreCase))
            return false;

        if (!filterHeightFrom.IsNullOrEmpty() && Convert.ToInt32(filterHeightFrom)
> form.Height)
            return false;

        if (!filterHeightTo.IsNullOrEmpty() && Convert.ToInt32(filterHeightTo) <
form.Height)
            return false;

        return true;
    }

```

MainWindow.xaml

```

<StackPanel>
    <Label FontSize="16">Фильтр по росту</Label>
    <Label>от</Label>
    <TextBox Text="{Binding filterHeightFrom,
UpdateSourceTrigger=PropertyChanged}" materialDesign:HintAssist.Hint="Введите
рост" TextChanged="TextBox_TextChanged"/>
    <Label>до</Label>
    <TextBox Text="{Binding filterHeightTo,
UpdateSourceTrigger=PropertyChanged}" materialDesign:HintAssist.Hint="Введите
рост" TextChanged="TextBox_TextChanged"/>
</StackPanel>

```

В результате можно фильтровать список по росту

Данные формы

Поиск  
Татьяна

Сортировка

Фильтры

Фильтр по росту

от  
170

до  
175

Сбросить

|                      |            |                   |          |            |
|----------------------|------------|-------------------|----------|------------|
| Федорова<br>Татьяна  | Возраст 47 | Год рождения 1978 | Рост 173 | Вес 91.72  |
| Матвеева<br>Татьяна  | Возраст 72 | Год рождения 1953 | Рост 174 | Вес 108.30 |
| Тарасова<br>Татьяна  | Возраст 68 | Год рождения 1957 | Рост 171 | Вес 95.87  |
| Белоусова<br>Татьяна |            |                   | Рост 172 | Вес 108.40 |

Всего записей: 10000 Показано: 12

Далее будет реализована сортировка. Для сортировки будет использован `CheckBox`. Для сортировки также используется `ICollectionView`. Для указания сортировки используется свойство интерфейса `SortDescription` через метод `Add` в него передается параметр сортировки (объект класса `SortDescription`). Будем передавать анонимный экземпляр. В конструкторе принимается два значения - поле сортировки и тип сортировки (`ListSortDicrection.Ascending` - по возрастанию или `ListSortDicrection.Descending` - по убыванию).

Реализовать сортировку нужно внимательно, так как `ICollectionView` позволяет сортировку по нескольким полям. В нужный момент сортировку необходимо сбрасывать.

Так как для указания сортировки используется `ComboBox` с предустановленными значениями реализуем для `ComboBox` событие `SelectionChanged`

MainWindows.xaml

```
<Label FontSize="16">Сортировка</Label>
<ComboBox SelectionChanged="ComboBox_SelectionChanged">
    <ComboBox.Items>
        <ComboBoxItem Tag="Age">По возрасту</ComboBoxItem>
        <ComboBoxItem Tag="Weight">По весу</ComboBoxItem>
        <ComboBoxItem Tag="Height">По росту</ComboBoxItem>
    </ComboBox.Items>
</ComboBox>
```

MainWindows.xaml.cs

```

private void ComboBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    formsView.SortDescriptions.Clear();
    var cb = (ComboBox)sender;
    var selected = (ComboBoxItem)cb.SelectedItem;
    switch (selected.Tag)
    {
        case "Age":
            formsView.SortDescriptions.Add(new SortDescription("Age",
ListSortDirection.Ascending));
            break;
        case "Weight":
            formsView.SortDescriptions.Add(new SortDescription("Weight",
ListSortDirection.Ascending));
            break;
        case "Height":
            formsView.SortDescriptions.Add(new SortDescription("Height",
ListSortDirection.Ascending));
            break;
    }
    formsView.Refresh();
}

```

В результате реализована сортировка используя ComboBox

Данные формы

Поиск  
Василиса

Сортировка  
По возрасту

Фильтры  
Фильтр по росту  
от  
170  
до  
175

Сбросить

|           |          |            |                   |          |            |
|-----------|----------|------------|-------------------|----------|------------|
| Никонова  | Василиса | Возраст 20 | Год рождения 2005 | Рост 172 | Вес 50.52  |
| Гаврилова | Василиса | Возраст 22 | Год рождения 2003 | Рост 173 | Вес 69.09  |
| Носова    | Василиса | Возраст 24 | Год рождения 2001 | Рост 170 | Вес 119.62 |
| Киселева  | Василиса |            |                   | Рост 174 | Вес 90.73  |

Всего записей: 10000 Показано: 13

## Задание

Вам необходимо разработать приложение для магазина электроники. Нужно реализовать функционал для двух ролей - посетитель и менеджер.

Посетитель может просматривать товары, выполнять поиск, фильтрацию и сортировку.

Фильтрация может осуществляться по следующим признакам:

- по категории
- по бренду
- по цене (от и до)

Сортировка товаров выполняется по следующим полям:

- Наименование
- Цена (убывание и возрастание)
- Количество (убывание и возрастание)

Менеджер может выполнять все те же операции что и пользователь, а также следующие операции:

- добавлять/просматривать/изменять/удалять товары
- добавлять/просматривать/изменять/удалять категории
- добавлять/просматривать/изменять/удалять теги
- добавлять/просматривать/изменять/удалять бренды товаров

Менеджер осуществляет вход в приложение по пин-коду 1234 при запуске приложения. В случае неверного ввода пин-кода, выдавать сообщение об ошибке.

Посетитель входит в приложение по нажатию кнопки "Войти как посетитель"

## Требования к оформлению приложения

Основной фон: #dfeaeб

Основной акцентный цвет: #00C4B4 (Кнопки, поля ввода, другие интерактивные элементы)

Дополнительный цвет: #00b4c4 (дополнительное выделение)

Все данные отображаемые на формах должны легко читаться.

Все экранные формы (окна/страницы) должны иметь заголовки.

Оформление приложения должно иметь единообразный вид в развернутом виде (на весь экран) и в режиме окна.



**Все элементы интерфейса должны быть отображены на экране без необходимости масштабировать окно (списки прокручивать можно).**

Элементы списка товаров магазина должны быть оформлены по следующему шаблону

|                   |          |
|-------------------|----------|
| Название товара   | Цена     |
| ★рейтинг          |          |
| Категория         |          |
| Описание          | Остаток: |
| #тег1 #тег2 #тег3 |          |

В случае если остаток товара меньше 10, то данный товар нужно выделить границей следующего цвета - #ffd129

На форме отображения товаров должно быть реализовано отображение количества товаров в базе данных и количество товаров отображенных в списке.

Данные должны валидироваться в соответствии с типами. Все поля обязательны для заполнения (за исключением связей многие-ко-многим).

Все необратимые действия должны быть подтверждены пользователем

## Приложения

Для реализации приложения вам необходимо реализовать базу данных на основании данных файла представленного по следующей ссылке - [Данные для импорта](#)

## Сдача проекта

Для сдачи проекта в репозитории должно находиться:

- ER-диаграмма базы данных, отображающая связи, первичные и внешние ключи и типы данных (в формате PDF)
- Скрипт восстановления базы данных (схема и данные)
- Реализованный проект